Algoritmos Constructivos para el Problema NWJSSP

Oscar Díaz

Universidad EAFIT

15 de marzo de 2026

Contenido

1. Introducción
   - Descripcion del Problema
   - Algoritmos
     - Heuristica Constructiva
     - Heuristica Aleatorizada 1 (NEH con Ruido)
     - Heuristica Aleatorizada 2 (NEH con GRASP)
2. Metodologia
3. Resultados
   - Exposición de Resultados
   - Cota inferior y medida de calidad
   - Resultados Z y tiempo de computo
   - Gap respecto a la cota inferior
   - Experimentos de Parámetros
4. Discusion
   - Analisis resultados
   - Conclusiones
5. Referencias

Descripcion del Problema

El **No-Wait Job Shop Scheduling Problem (NWJSSP)** consiste en programar un conjunto de n trabajos sobre un conjunto de m máquinas, minimizando el tiempo de flujo total $Z = \sum_j C_j$, donde C_j es el tiempo de finalización del trabajo j.

Cada trabajo j posee:

- Una **fecha de liberación** r_j: instante más temprano en que puede comenzar a procesarse.
- Una **secuencia fija de** m **operaciones**, donde la operación u requiere la máquina m_{ju} con un tiempo de procesamiento p_{ju}.

Las restricciones del problema son:

1. Cada operación u de un trabajo solo puede iniciar tras completar la operación $u - 1$.
2. Una vez iniciada, ninguna operación puede interrumpirse.
3. Cada máquina procesa a lo sumo un trabajo a la vez.
4. **Restricción No-Wait**: no se permite tiempo de espera entre operaciones consecutivas del mismo trabajo

Heuristica Constructiva (NEH Adaptado)

El algoritmo constructivo implementado es una adaptación de la heurística **NEH** (Nawaz, Enscore y Ham, 1983) al NWJSSP.

Paso 1 Orden inicial

Para cada trabajo j se calcula el tiempo minimo de procesamiento $r_j + \sum_u p_{ju}$, que combina su fecha de liberación con su carga total de procesamiento. Los trabajos se ordenan de **mayor a menor** por este tiempo minimo de procesamiento, con el pretexto de que los trabajos que se demoran mas tienen mayor prioridad a la hora de ser insertados.

Paso 2 Inserción iterativa

Los trabajos se procesan en el orden mencionado en Paso 1. Para cada trabajo j se busca la **mejor posición** dentro de la secuencia parcial ya construida, probando todas las posiciones disponibles $\{0, 1, \ldots, |\text{secuencia}|\}$.

Heuristica Constructiva (NEH Adaptado)

Paso 3 Evaluación de posiciones por bloques temporizados
Evaluar todas las posiciones en instancias grandes era costoso y demorado computacionalmente. Para mejorar lo anterior, las posiciones se agrupan en **bloques de tamaño** $b = \text{máx}(10, \lfloor\sqrt{n}\rfloor)$. Cada bloque tiene un de tiempo máximo de procesamiento $t_{\text{máx}}$: si se agota dentro del bloque, ese bloque se interrumpe y se avanza al siguiente. Todos los bloques son visitados siempre, garantizando cobertura global de la secuencia.

Paso 4 Criterio de evaluación
Para cada posición candidata se calcula el **Total Flow Time** $Z = \sum_j C_j$ que resultaría de insertar j allí, simulando la secuencia parcial completa.

Paso 5 Inserción
Al recorrer todos los bloques, el trabajo j se inserta en la posición de **menor impacto sobre** Z. El proceso se repite hasta que todos los trabajos han sido ubicados en la secuencia.

Heuristica Aleatorizada 1 (NEH con Ruido)

El primer método aleatorizado es una extensión del NEH adaptado basada en **construcciones con ruido**. La idea es que el NEH puro siempre produce la misma solución al tener un ordenamiento inicial determinístico. Para explorar distintos puntos de partida, se perturba dicho ordenamiento mediante ruido en cada iteración.

Paso 1 Ordenamiento con ruido

Para cada trabajo j se calcula su tiempo mínimo total de procesamiento $w_j = r_j + \sum_u p_{ju}$ y se reemplaza por un peso con ruido:

$$w_j^{ruido} = w_j + \rho_j, \qquad \rho_j \sim U(-R \cdot w_j,\ +R \cdot w_j)$$

donde R controla la amplitud del ruido de forma **proporcional** a w_j. Los trabajos se ordenan de mayor a menor por w_j^{ruido}. Una vez generado este orden perturbado, **el proceso de construcción se vuelve determinista**, siguiendo los mismos Pasos 2 al 5 del NEH constructivo.

Heuristica Aleatorizada 1 (NEH con Ruido

Construcciones independientes

El proceso completo se repite `N_ITER` veces de forma independiente, generando un ordenamiento con ruido distinto en cada iteración. La primera iteración usa $R = 0$, reproduciendo el NEH puro como referencia. Las iteraciones siguientes usan $R > 0$, explorando ordenamientos alternativos. Al finalizar se retiene la secuencia con menor Z:

$$Z^* = \text{mín}(Z_0,\, Z_1,\, \ldots,\, Z_{\texttt{N_ITER}-1})$$

Heuristica Aleatorizada 2 (NEH con GRASP)

El segundo método aleatorizado implementa la fase de construcción de **GRASP** (Greedy Randomized Adaptive Search Procedures, Feo y Resende, 1989) integrada en el marco NEH. A diferencia del NEH puro, en lugar de seleccionar siempre el trabajo de mayor w_j para insertar, se construye en cada paso una **Lista Restringida de Candidatos (RCL)** de la cual se selecciona un trabajo aleatoriamente.

Construcción de la RCL basada en valor

En cada paso, con los trabajos aún pendientes de insertar, se calcula $w_j = r_j + \sum_u p_{ju}$ para cada candidato y se construye la RCL incluyendo solo los trabajos cuyo w_j cae dentro del rango:

$$\text{RCL} = \left\{ j \in \text{pendientes} \;\middle|\; w_j \geq w_{max} - \alpha \cdot (w_{max} - w_{min}) \right\}$$

Heuristica Aleatorizada 2 (NEH con GRASP)

Carácter adaptativo

La RCL se recalcula en **cada paso** usando únicamente los trabajos todavía pendientes. Esto significa que w_{max} y w_{min} cambian en cada iteración interna. Lo cual es el carácter *adaptativo* del método: los criterios de selección se actualizan con cada decisión tomada.

Selección e inserción

Se selecciona aleatoriamente con probabilidad uniforme un trabajo j^* de la RCL y se inserta en la mejor posición de la secuencia parcial, siguiendo los mismos Pasos 2 al 5 del NEH constructivo. El proceso se repite hasta insertar todos los trabajos.

Construcciones independientes

El proceso completo se repite `N_ITER` veces. La primera iteración usa $\alpha = 0$ como referencia. Las siguientes usan $\alpha > 0$, explorando distintas secuencias. Se retiene la de menor Z:

$$Z^* = \text{mín}(Z_0, Z_1, \ldots, Z_{\texttt{N_ITER}-1})$$

Metodología

1. **Implementación de algoritmos en Python:** Se programaron tres algoritmos, un método constructivo (NEH adaptado) y dos métodos aleatorizados (NEH con Ruido y Construcción GRASP), aplicados al NWJSSP.
2. **Ejecución sobre instancias y exportación de resultados:** Cada algoritmo recorre automáticamente todas las instancias disponibles y exporta los resultados en el formato requerido a un archivo Excel. Los métodos aleatorizados ejecutan 10 iteraciones independientes por instancia, reteniendo la solución de menor Z.
3. **Cálculo de cota inferior teórica:** Se calcula una cota inferior para cada instancia que permite cuantificar la calidad de las soluciones obtenidas mediante un GAP.
4. **Análisis de parámetros:** Se ejecutan simulaciones con distintos valores de los parámetros de cada método para posterior analisis.
5. **Comparación y análisis de resultados:** Se comparan los tres métodos entre sí en términos de valor de Z, GAP y tiempo de cómputo.

Exposición de Resultados

Los resultados se presentan en el siguiente orden. Primero se muestra la **cota inferior teórica** (LB) calculada para cada instancia, que sirve como referencia para medir la calidad de las soluciones. Luego se presentan los **resultados principales** de los tres métodos, valor de la función objetivo Z y tiempo de cómputo t_c (en ms), junto con el **GAP** relativo respecto a la cota inferior:

$$\text{GAP} = \frac{Z - LB}{LB}$$

Finalmente se presentan los **experimentos de parámetros** fueron realizados para las instancias con trabajos inferiores a 1000 por coste computacional: para cada método se varía un parámetro clave manteniendo los demás fijos, con el objetivo de identificar la configuración que produce los mejores valores de Z.

Cota inferior y medida de calidad

Cota inferior (LB) Se obtiene relajando las restricciones del problema *NWJSSP*:

- Cada trabajo inicia en su tiempo de liberación r_j
- No existen conflictos de máquinas
- Las operaciones pueden ejecutarse simultáneamente

Bajo estas hipótesis:

$$LB = \sum_{j=1}^{n} (r_j + P_j)$$

$$P_j = \sum_{u=1}^{m} p_{ju}$$

Consistencia de la cota En el problema original se cumple:

$$C_j \geq r_j + P_j$$

Por lo tanto:

$$\sum_{j=1}^{n} C_j \geq LB$$

A continuacion se presentan las cotas inferiores LB calculadas para cada instancia, asi mismo se especifica un indice para cada instancia y en las siguientes presentaciones no se repite el nombre de las instancias

Cotas inferiores

Índice	Instancia	n	m	LB
1	ft06	6	6	197
2	ft06r	6	6	283
3	ft10	10	10	5109
4	ft10r	10	10	6055
5	ft20	20	5	5109
6	ft20r	20	5	6384
7	tai_j10_m10_1	10	10	52755
8	tai_j10_m10_1r	10	10	61293
9	tai_j100_m10_1	100	10	504826
10	tai_j100_m10_1r	100	10	595659
11	tai_j100_m100_1	100	100	4998668
12	tai_j100_m100_1r	100	100	6146561
13	tai_j1000_m10_1	1000	10	5021370
14	tai_j1000_m10_1r	1000	10	6323675
15	tai_j1000_m100_1	1000	100	50035544
16	tai_j1000_m100_1r	1000	100	54669545

Resultados Z y tiempo de computo

Instancia			neh_basic		neh_simple_noise		neh_grasp	
indice	n	m	Z	tc	Z	tc	Z	tc
1	6	6	464	0	452	1	464	1
2	6	6	486	0	486	1	486	1
3	10	10	13,945	1	13,878	7	13,878	8
4	10	10	14,800	1	14,478	8	14,640	8
5	20	5	22,179	3	21,787	33	21,904	38
6	20	5	24,577	3	22,589	33	23,006	36
7	10	10	185,639	1	177,941	7	179,649	12
8	10	10	195,538	1	183,787	8	183,787	12
9	100	10	11,863,358	618	11,309,222	6,059	11,301,112	6,646
10	100	10	11,919,428	631	11,332,263	6,018	11,371,981	7,147
11	100	100	193,131,578	4,243	192,063,054	41,494	193,150,710	51,252
12	100	100	192,539,782	4,234	192,271,800	41,557	193,551,496	51,406
13	1000	10	1,029,501,567	176,218	998,073,297	1,716,576	1,038,089,050	1,808,738
14	1000	10	1,010,266,581	176,257	980,732,517	1,718,332	1,025,553,435	2,196,014
15	1000	100	19,598,088,625	228,995	19,257,996,516	2,225,006	19,344,219,524	3,443,733
16	1000	100	19,477,815,061	245,769	19,246,174,599	2,223,411	19,345,106,523	2,463,718

Gap respecto a la cota inferior

Instancia			neh_basic		neh_simple_noise		neh_grasp	
indice	n	m	Gap	tc	Gap	tc	Gap	tc
1	6	6	1.36	0	1.29	1	1.36	1
2	6	6	0.72	0	0.72	1	0.72	1
3	10	10	1.73	1	1.72	7	1.72	8
4	10	10	1.44	1	1.39	8	1.42	8
5	20	5	3.34	3	3.26	33	3.29	38
6	20	5	2.85	3	2.54	33	2.60	36
7	10	10	2.52	1	2.37	7	2.41	12
8	10	10	2.19	1	2.00	8	2.00	12
9	100	10	22.50	618	21.40	6059	21.39	6646
10	100	10	19.01	631	18.02	6018	18.09	7147
11	100	100	37.64	4243	37.42	41494	37.64	51252
12	100	100	30.32	4234	30.28	41557	30.49	51406
13	1000	10	204.02	176218	197.77	1716576	205.73	1808738
14	1000	10	158.76	176257	154.09	1718332	161.18	2196014
15	1000	100	390.68	228995	383.89	2225006	385.61	3443733
16	1000	100	355.28	245769	351.05	2223411	352.86	2463718

Experimento 1 NEH Básico: variación de TIME_LIMIT

Instancia			$t_{\text{máx}} = 0{,}001$		$t_{\text{máx}} = 0{,}005$		$t_{\text{máx}} = 0{,}01$		$t_{\text{máx}} = 0{,}05$		$t_{\text{máx}} = 0{,}1$	
idx	n	m	Z	t_c	Z	t_c	Z	t_c	Z	t_c	Z	t_c
1	6	6	464	0	464	0	464	0	464	0	464	0
2	6	6	486	0	486	0	486	0	486	0	486	0
3	10	10	13,945	1	13,945	1	13,945	1	13,945	1	13,945	1
4	10	10	14,800	1	14,800	1	14,800	1	14,800	1	14,800	1
5	20	5	22,179	5	22,179	7	22,179	5	22,179	4	22,179	4
6	20	5	24,577	3	24,577	5	24,577	3	24,577	3	24,577	3
7	10	10	185,639	1	185,639	1	185,639	1	185,639	1	185,639	1
8	10	10	195,538	1	195,538	1	195,538	1	195,538	1	195,538	1
9	100	10	11,879,138	491	11,863,358	622	11,863,358	615	11,863,358	631	11,863,358	610
10	100	10	12,122,323	485	11,919,428	606	11,919,428	623	11,919,428	618	11,919,428	615
11	100	100	201,059,572	724	194,552,073	2,669	192,968,333	4,164	192,209,685	4,432	192,209,685	4,444
12	100	100	202,125,591	730	195,536,190	2,686	192,228,666	4,201	192,169,680	4,354	192,169,680	4,415

Experimento 2 NEH con Ruido: variación de R

Instancia			$R = 0{,}00$		$R = 0{,}05$		$R = 0{,}10$		$R = 0{,}15$		$R = 0{,}25$		$R = 0{,}40$	
idx	n	m	Z	t_c	Z	t_c	Z	t_c	Z	t_c	Z	t_c	Z	t_c
1	6	6	464	1	464	1	464	1	452	1	452	1	435	1
2	6	6	486	1	486	1	486	1	486	1	472	1	472	1
3	10	10	13,945	8	13,878	7	13,878	8	13,713	8	13,870	8	13,870	8
4	10	10	14,800	8	14,640	8	13,628	8	13,628	8	13,628	8	13,628	8
5	20	5	22,179	34	21,730	33	21,730	32	21,730	32	21,186	39	21,186	35
6	20	5	24,577	33	22,900	33	21,907	33	22,589	32	22,306	33	22,556	32
7	10	10	185,639	7	179,649	8	179,649	8	185,639	8	175,008	8	175,008	8
8	10	10	195,538	8	183,787	8	183,787	8	183,787	8	175,008	8	174,358	8
9	100	10	11,863,358	6,085	11,512,791	6,135	11,357,380	6,128	11,319,567	6,142	11,264,829	6,243	11,266,071	6,587
10	100	10	11,919,428	7,785	11,555,681	7,340	11,164,278	7,326	11,379,083	7,427	11,221,498	7,547	11,225,113	7,737
11	100	100	192,222,044	45,721	193,256,851	45,205	192,878,992	43,762	192,290,474	45,285	192,091,701	46,676	192,813,257	46,614
12	100	100	192,423,478	45,672	192,620,880	46,431	192,737,164	44,702	192,343,545	45,952	192,827,575	46,298	192,325,093	44,877

Experimento 3 GRASP: variación de α (ALPHA)

Instancia			$\alpha = 0{,}00$		$\alpha = 0{,}05$		$\alpha = 0{,}10$		$\alpha = 0{,}20$		$\alpha = 0{,}35$		$\alpha = 0{,}50$	
idx	n	m	Z	t_c	Z	t_c	Z	t_c	Z	t_c	Z	t_c	Z	t_c
1	6	6	464	1	464	1	464	1	464	2	464	1	452	1
2	6	6	486	1	486	2	486	1	486	2	486	3	486	4
3	10	10	13,945	9	13,945	9	13,945	8	13,878	8	13,878	8	13,945	8
4	10	10	14,800	8	14,640	8	14,640	8	14,640	8	13,628	8	13,628	8
5	20	5	22,179	34	22,011	33	21,127	33	21,730	33	21,852	33	21,943	34
6	20	5	24,577	33	23,760	33	22,787	33	22,796	33	22,962	34	21,146	33
7	10	10	185,639	8	179,649	8	179,649	8	185,639	8	185,639	8	185,639	8
8	10	10	195,538	8	195,538	8	183,787	8	183,787	8	185,639	8	179,649	8
9	100	10	11,863,358	6,180	11,465,043	6,137	11,379,224	6,117	11,402,598	6,159	11,281,836	6,121	11,324,538	6,235
10	100	10	11,919,428	6,129	11,482,505	6,157	11,473,871	8,317	11,327,818	8,240	11,372,016	8,569	11,167,747	6,840
11	100	100	191,910,621	42,358	192,258,634	43,719	192,366,688	42,665	192,702,056	43,644	192,258,634	42,787	192,583,386	42,643
12	100	100	192,343,545	42,774	192,957,786	43,723	192,454,228	42,944	192,089,393	42,506	192,454,228	42,579	191,907,107	42,401

Analisis resultados (1)

- **Resultados Z y tiempo de computo:** En la tabla de resultados principales se observa que el NEH con Ruido logra consistentemente mejores valores de Z que el NEH con GRASP en la mayoría de instancias, lo cual es contraintuitivo dado que GRASP tiene un mecanismo de aleatorización más sofisticado.
- **GAP respecto a la cota inferior:** En la tabla de GAPs se evidencia con claridad que a medida que crece n, la distancia relativa entre la solución obtenida y la cota inferior crece de forma pronunciada, prácticamente exponencial.

Analisis resultados (2)

- **Cambio de parámetros Para el NEH básico:** el experimento de TIME LIMIT muestra que valores de 0.005 o superiores producen resultados idénticos en instancias pequeñas y medianas, mientras que para instancias con n=100 y m=100 el umbral de 0.01 segundos ya alcanza la estabilidad.
- **Cambio de parámetros Para el NEH con ruido:** los resultados mejoran progresivamente al aumentar R hasta valores cercanos a 0.25, donde se obtienen los mejores Z en la mayoría de instancias medianas; valores más altos como R=0.40 no aportan ganancias consistentes y pueden degradar la solución en algunas instancias
- **Elección de parámetros Para el NEH GRASP:** el parámetro α no muestra un patrón tan claro: valores bajos (0.05, 0.10) son suficientes para instancias pequeñas mientras que para instancias medianas valores más altos (0.35, 0.50) en ocasiones producen mejores resultados.

Conclusiones

- **Valor de los métodos aleatorizados.** La exploración de múltiples soluciones mediante perturbación del ordenamiento inicial demuestra ser una estrategia efectiva para mejorar la solución del NEH básico sin incrementar significativamente la complejidad algorítmica. En particular, el NEH con Ruido supera al GRASP en la mayoría de instancias, sugiriendo que diversificar el punto de partida es más valioso que restringir la selección mediante una RCL para el NWJSSP.
- **Escalabilidad computacional.** El costo computacional de la generación iterativa de soluciones crece de forma muy agresiva con el tamaño de la instancia, alcanzando tiempos superiores a 30 minutos para $n = 1000$ con 10 iteraciones. Esto limita severamente la aplicabilidad práctica del enfoque aleatorizado en instancias grandes y motiva la integración con metaheurísticas que exploren el espacio de soluciones de forma más dirigida.

Frame Title

- **Limitación en la evaluación de factibilidad.** La función `find_start()` calcula el inicio factible de un trabajo asumiendo que cada máquina está ocupada de forma continua desde el tiempo 0 hasta su último instante de uso, ignorando los intervalos libres que pudieran existir entre trabajos ya programados. Esto implica que las heurísticas no insertan trabajos en ventanas disponibles dentro del schedule, produciendo soluciones subóptimas donde se desaprovechan huecos de tiempo libre, una limitación estructural que afecta la calidad de los tres métodos implementados.

Referencias

Nawaz, M., Enscore, E. E., & Ham, I. (1983).
A heuristic algorithm for the *m*-machine, *n*-job flow-shop sequencing problem.
Omega, 11(1), 91–95.

Feo, T. A., & Resende, M. G. C. (1989).
A probabilistic heuristic for a computationally difficult set covering problem.
Operations Research Letters, 8(2), 67–71.